



Trabalho Prático 2 - Processamento de Imagens PNM

..... **INF110**

Rafael Vagner Pinto da Fonseca Souza - Matrícula: 124680



Na criação/desenvolvimento do código deste trabalho, utilizei como base o arquivo **TP2-codbase.cpp**, que está disponível no PVANet Moodle.

Resumidamente, meu programa ficou dividido em:

- Declarações + Leitura da imagem
- Leitura do cabeçalho da imagem
- Leitura dos pixels da imagem
- Escolha da operação de processamento da imagem
- Nove operações
- Gravação da imagem

● ● ● ● ● Nove operações ● ● ● ● ●

- 1 - Escurecer
- 2 - Clarear
- 3 - Gerar o negativo
- 4 - Espelhar Horizontalmente
- 5 - Espelhar Verticalmente
- 6 - Aplicar filtro de sobel
- 7 - Aplicar filtro de realce (sharpening)
- 8 - Aplicar filtro de binarizacao
- 9 - Transformar a imagem colorida em tons de cinza

Declarações + Leitura da imagem

```
#include <iostream>
#include <fstream>
#include <cstring>
#include <cmath>

const int MAXALTURA = 2000;    //tamanho maximo de imagem aceito
const int MAXLARGURA = 2000;

using namespace std;
unsigned char imagem[MAXALTURA][MAXLARGURA];    //a matriz para imagem do tipo P2
unsigned char imagemP3[MAXALTURA][MAXLARGURA][3];    //a matriz para imagem do tipo P3
int matauxP2[MAXALTURA+2][MAXLARGURA+2];    //matriz auxiliarr pra imagens P2
int matauxP3[MAXALTURA+2][MAXLARGURA+2][3];    //matriz auxiliar pra imagens P3
int main() {
    int largura, altura;    //dimensoes da imagem
    char tipo[4];    //tipo da imagem
    ifstream arqentrada;    //arquivo que contem a imagem original
    ofstream arqsaida;    //arquivo que contera a imagem modificada
    char comentario[200], c;    //auxiliares
    int i, j, valor, p;    //auxiliares

    /*** LEITURA DA IMAGEM ***/
    /*** Abertura do arquivo ***/

    char nomearq[100];
    cout << "Digite o nome da imagem: ";
    cin >> nomearq;
    arqentrada.open(nomearq, ios::in);    //Abre arquivo para leitura
    if (!arqentrada) {
        cout << "Nao consegui abrir arquivo imagem.pnm" << endl;
        return 0;
    }
}
```

Declarações

Neste trecho é definido a altura e largura máxima aceita da imagem. Juntamente é declarado matrizes auxiliares utilizadas, bibliotecas e algumas outras variáveis.

Algumas variáveis são declaradas como unsigned char. Este tipo de dado é ideal nesse caso, pois ocupa só um byte e armazena valores de 0 a 255, ou seja, é compatível com o padrão de representação de pixels em imagens.

Leitura da imagem

Neste trecho, o programa solicita o nome do arquivo da imagem e o procura na pasta do programa. Caso o arquivo exista, ele é aberto; caso contrário, é retornado um erro.

Leitura do cabeçalho da imagem

```
/***/ Leitura do cabeçalho ***/
arqentrada >> tipo;      //Le o tipo de arquivo
arqentrada.get();        //Le e descarta o \n do final da 1a. linha

if (strcmp(tipo,"P2")==0) {
    cout << "Imagem em tons de cinza\n";
}
else if (strcmp(tipo,"P3")==0) {
    cout << "Imagem colorida\n";
}
else if (strcmp(tipo,"P1")==0) {
    cout << "Imagem preto e branco\n";
    cout << "Desculpe, nao trabalho com esse tipo de imagem.\n";
    arqentrada.close();
    return 0;
}
else if (strcmp(tipo,"P4")==0 || strcmp(tipo,"P5")==0 || strcmp(tipo,"P6")==0) {
    cout << "Imagem no formato RAW\n";
    cout << "Desculpe, nao trabalho com esse tipo de imagem.\n";
    arqentrada.close();
    return 0;
}
while((c = arqentrada.get()) == '#')      //Enquanto for comentario
    arqentrada.getline(comentario,200); //Le e descarta a linha "inteira"

arqentrada.putback(c);      //Devolve o caractere lido para a entrada, pois como
                           //nao era comentario, era o primeiro digito da largura
arqentrada >> largura >> altura;      //Le as dimensoes da imagem, numero de pixels da horizontal e da vertical
cout << "Tamanho: " << largura << " x " << altura << endl;
if (largura > MAXLARGURA) {
    cout << "Desculpe, ainda nao trabalho com imagens com mais de " << MAXLARGURA << " pixels de largura.\n";
    arqentrada.close();
    return 0;
}
if (altura > MAXALTURA) {
    cout << "Desculpe, ainda nao trabalho com imagens com mais de " << MAXALTURA << " pixels de altura.\n";
    arqentrada.close();
    return 0;
}
arqentrada >> valor;      //Valor maximo do pixel (temos que ler, mas nao sera usado, assumimos 255)
```

Na leitura do cabeçalho, o programa lê as informações da imagem selecionada para identificar seu tipo de arquivo e descartar comentários que não são relevantes.

Dessa forma, este trecho aceita apenas imagens dos tipos P2 e P3, correspondentes a tons de cinza e coloridas, respectivamente.

Caso a imagem não seja do tipo P2 ou P3, ou caso suas dimensões ultrapassem os valores máximos definidos, o programa retorna um erro.

Leitura dos pixels da imagem

```
/**** Leitura dos pixels da imagem ***/

//PARA IMAGEM DO TIPO P2
if(strcmp(tipo,"P2")==0) {
    for(i=0;i<altura;i++)
        for(j=0;j<largura;j++) {
            arqentrada >> valor;
            imagem[i][j] = (unsigned char)valor;    //guarda no formato unsigned char
        }
}

//PARA IMAGEM DO TIPO P3
else if(strcmp(tipo,"P3")==0) {
    for(i=0;i<altura;i++)
        for(j=0;j<largura;j++) {
            for(p=0; p<3; p++){
                arqentrada >> valor;
                imagemP3[i][j][p] = (unsigned char)valor;    //guarda no formato unsigned char
            }
        }
}

arqentrada.close(); //Fecha arquivo apos a leitura

/**** Fim da leitura da imagem ***/
```

OBS: A partir deste trecho do programa começa a ter separação do código para cada tipo de imagem, P2 e P3.

Nessa parte de leitura dos pixels da imagem, o programa lê os valores que formam cada pixel da imagem e os armazena na matriz *imagem*, caso a imagem seja do tipo P2, ou na matriz *imagemP3*, caso seja do tipo P3 .

Ambas as matrizes foram declaradas no início do programa. Após a leitura completa dos dados da imagem, o arquivo é fechado.

Escolha da operação de processamento da imagem

```
/** Escolha da operação de processamento da imagem */  
  
int options;  
cout << "Oi! O que deseja fazer com a imagem?" << endl;  
cout << "1 - Escurecer" << endl;  
cout << "2 - Clarear" << endl;  
cout << "3 - Gerar o negativo" << endl;  
cout << "4 - Espelhar Horizontalmente" << endl;  
cout << "5 - Espelhar Verticalmente" << endl;  
cout << "6 - Aplicar filtro de sobel" << endl;  
cout << "7 - Aplicar filtro de realce (sharpening)" << endl;  
cout << "8 - Aplicar filtro de binarizacao" << endl;  
cout << "9 - Transformar a imagem colorida em tons de cinza" << endl;  
  
cin >> options;  
if(options < 1 || options > 9){  
    cout << "Opção inválida!" << endl;  
    return 0;  
}
```

Na escolha da operação, são exibidas todas as ações de processamento disponíveis para serem aplicadas na imagem.

Neste trecho do código, é solicitado ao usuário que escolha, de 1 a 9, qual operação deseja executar.

A opção selecionada vai definir o fluxo de execução do programa, direcionando-o para uma das nove operações, por meio de estrutura condicional (if e else)

1 - Escurecer

```
/** Escurecer imagem */
if(options ==1){
    int fatorE;
    cout << "Qual o fator de escurecimento (0-255)? ";
    cin >> fatorE;
    if(fatorE < 0)
        fatorE = 0; // proteção

//IMAGEM TIPO P2
if(strcmp(tipo,"P2")==0) {
    for(i=0;i<altura;i++)
        for(j=0;j<largura;j++) {
            valor = (int)imagem[i][j]; //pega o valor do pixel
            valor -= fatorE; //escurece o pixel em certo valor (fatorE)
            if (valor < 0) //se der negativo iguala a 0 (preto)
                valor = 0;
            if (valor > 255) //se o valor for maior que o valor máximo, iguala ao valor máximo
                valor = 255;
            imagem[i][j] = (unsigned char)valor; //modifica o pixel
        }
    }
//IMAGEM TIPO P3
else if(strcmp(tipo,"P3")==0) {
    for(i=0;i<altura;i++)
        for(j=0;j<largura;j++) {
            for(p=0; p<3; p++){
                valor = (int)imagemP3[i][j][p]; //pega o valor das bandas de cores dos pixels
                valor -= fatorE; //escurece em certo valor(fatorE)
                if (valor < 0) //se der negativo iguala a 0
                    valor = 0;
                if (valor > 255) //se o valor for maior que o valor máximo, iguala ao valor máximo
                    valor = 255;
                imagemP3[i][j][p] = (unsigned char)valor; //modifica o pixel
            }
        }
    }
}
```

Essa parte do código realiza a operação de escurecimento da imagem. É solicitado ao usuário um fator de escurecimento a ser aplicado.

Esse valor é subtraído do valor de cada pixel, no caso de imagens do tipo P2, e subtraído de cada componente de cor (RGB), no caso de imagens do tipo P3.

Além disso, o código possui proteções para garantir que o fator não seja negativo e que os valores resultantes dos pixels permaneçam no intervalo válido de 0 a 255.

1 - Escurecer - Exemplos de utilização



Imagem tipo P2



Imagem tipo P2 com fator de escurecimento de 150



Imagem tipo P3

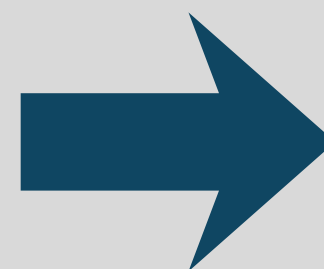


Imagem tipo P3 com fator de escurecimento de 150

2 - Clarear

```
/***/ Clarear a imagem ***/

else if(options ==2){
    int fatorC;
    cout << "Qual o fator de clareamento (0-255)? ";
    cin >> fatorC;
    if(fatorC < 0)
        fatorC = 0; // proteção

    //IMAGEM TIPO P2
    if(strcmp(tipo,"P2")==0) {
        for(i=0;i<altura;i++)
            for(j=0;j<largura;j++) {
                valor = (int)imagem[i][j]; //pega o valor do pixel
                valor += fatorC; //clareia o pixel em certo valor(fatorC)
                if (valor > 255) //se o valor for maior que o valor máximo, iguala ao valor máximo
                    valor = 255;
                imagem[i][j] = (unsigned char)valor; //modifica o pixel
            }
    }

    //IMAGEM TIPO P3
    else if(strcmp(tipo,"P3")==0) {
        for(i=0;i<altura;i++)
            for(j=0;j<largura;j++) {
                for(p=0; p<3; p++){
                    valor = (int)imagemP3[i][j][p]; //pega o valor das bandas de cores dos pixels
                    valor += fatorC; //clareia o pixel em certo valor(fatorC)
                    if (valor > 255) //se o valor for maior que o valor máximo, iguala ao valor máximo
                        valor = 255;
                    imagemP3[i][j][p] = (unsigned char)valor; //modifica o pixel
                }
            }
    }
}
```

O código que realiza a operação de clareamento da imagem é análogo ao código que realiza o escurecimento.

Porém, neste caso, é solicitado um fator de clareamento, que é somado ao valor de cada pixel, no caso de imagens do tipo P2, e somado ao valor de cada componente de cor (RGB), no caso de imagens do tipo P3.

Da mesma forma, são aplicadas as mesmas proteções citadas anteriormente para garantir que os valores permaneçam no intervalo válido de 0 a 255.

2 - Clarear - Exemplos de utilização



Imagem tipo P2

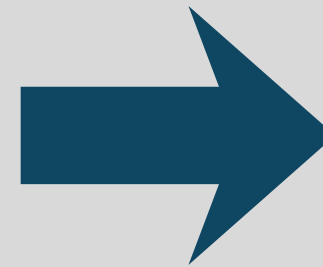


Imagem tipo P2 com fator de clareamento de 130



Imagem tipo P3

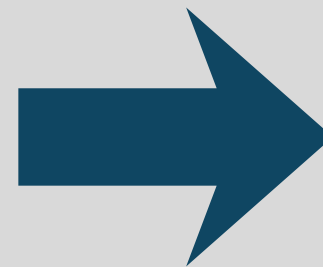


Imagem tipo P3 com fator de clareamento de 130

3 - Negativo

```
/** Gerar o negativo da imagem */  
  
else if(options ==3){  
  
//IMAGEM TIPO P2  
if(strcmp(tipo,"P2")==0) {  
    for(i=0;i<altura;i++)  
        for(j=0;j<largura;j++) {  
            valor = (int)imagem[i][j];    //pega o valor do pixel  
            valor = 255 - valor;           //inverte o valor do pixel  
            imagem[i][j] = (unsigned char)valor;    //modifica o pixel  
        }  
    }  
  
//IMAGEM TIPO P3  
else if(strcmp(tipo,"P3")==0) {  
    for(i=0;i<altura;i++)  
        for(j=0;j<largura;j++) {  
            for(p=0; p<3; p++){  
                valor = (int)imagemP3[i][j][p];    //pega o valor das bandas de cores dos pixels  
                valor = 255 - valor;                //inverte o valor de cada banda de cor do pixel  
                imagemP3[i][j][p] = (unsigned char)valor;    //modifica o pixel  
            }  
        }  
    }  
}
```

Ao selecionar a opção 3, esse trecho de código gera o negativo da imagem.

No caso de imagens do tipo P2, cada pixel é transformado por meio da operação $255 - \text{valor}$, resultando na inversão dos tons de cinza.

Já no caso de imagens do tipo P3, a mesma operação é aplicada a cada componente de cor (R, G e B) de cada pixel, produzindo a imagem negativa.

3 - Negativo - Exemplos de utilização



Imagem tipo P2



Imagem tipo P2 negativa



Imagem tipo P3

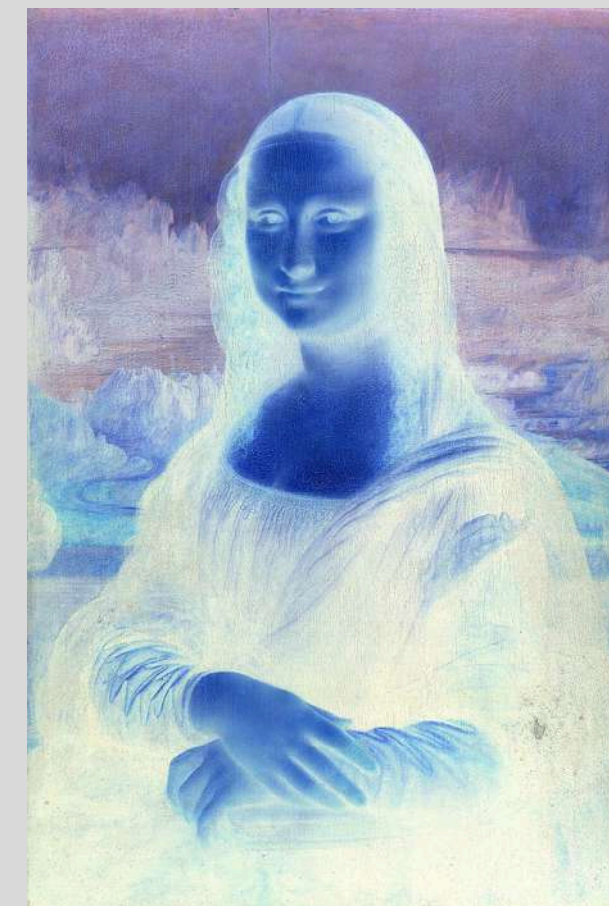
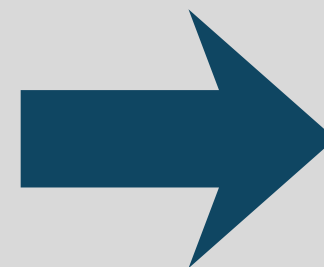


Imagem tipo P3 negativa

4 - Espelhamento Horizontal

```
/** Espelhar a imagem horizontalmente */
else if(options ==4){

//IMAGEM TIPO P2
if(strcmp(tipo,"P2")==0) {
    for(i=0;i<altura;i++){
        for(j=0;j<largura/2;j++){
            unsigned char auxl = imagem[i][j]; //troca o pixel de uma coluna
            imagem[i][j] = imagem[i][largura-1-j]; //pelo pixel da coluna simétrica
            imagem[i][largura-1-j] = auxl; //na mesma linha(i)
        }
    }
//IMAGEM TIPO P3
else if(strcmp(tipo,"P3")==0) {
    for(i=0;i<altura;i++){
        for(j=0;j<largura/2;j++){
            for(p=0; p<3; p++){
                unsigned char auxl = imagemP3[i][j][p]; // troca o pixel de uma coluna pelo pixel da coluna simétrica
                imagemP3[i][j][p] = imagemP3[i][largura-1-j][p]; // na mesma linha (i), mantendo os três canais de cor (RGB)
                imagemP3[i][largura-1-j][p] = auxl;
            }
        }
    }
}
}
```

Esse trecho de código, referente à opção 4, realiza o espelhamento horizontal da imagem.

No caso de imagens do tipo P2, a matriz *imagem* é percorrida e cada pixel de uma coluna é trocado com o pixel da coluna simétrica na mesma linha.

Já no caso de imagens do tipo P3, a matriz *imagemP3* é percorrida e cada pixel (com seus três componentes de cor) é trocado com o pixel da posição simétrica/oposta na mesma linha, realizando o espelhamento da imagem colorida.

Ou seja, nesta operação a 1º coluna é trocada pela última, a 2º pela penúltima...

4 - Espelhamento Horizontal - Exemplos de utilização



Imagem tipo P2

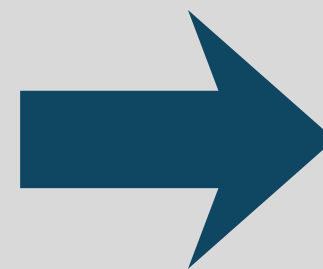


Imagem tipo P2 espelhada horizontalmente



Imagem tipo P3

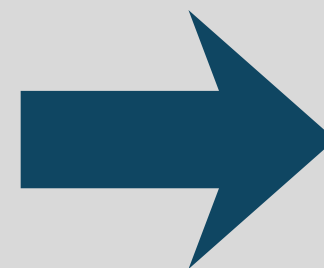


Imagem tipo P3 espelhada horizontalmente

5 - Espelhamento Vertical

```
/** Espelhar a imagem verticalmente**/

else if(options ==5){

//IMAGEM TIPO P2
if(strcmp(tipo,"P2")==0) {
    for(i=0;i<altura/2;i++){
        for(j=0;j<largura;j++){
            unsigned char auxl = imagem[i][j]; //troca o pixel de uma linha
            imagem[i][j] = imagem[altura - 1 - i][j]; //pelo pixel da linha simétrica
            imagem[altura - 1 - i][j] = auxl; //na mesma coluna(j)
        }
    }
//IMAGEM TIPO P3
else if(strcmp(tipo,"P3")==0) {
    for(i=0;i<altura/2;i++){
        for(j=0;j<largura;j++){
            for(p=0; p<3; p++){
                unsigned char auxl = imagemP3[i][j][p]; // troca o pixel (RGB) da linha atual com o pixel
                imagemP3[i][j][p] = imagemP3[altura-1-i][j][p]; // da linha simétrica, na mesma coluna (j),
                imagemP3[altura-1-i][j][p] = auxl;
            }
        }
    }
}
```

De forma análoga, a operação de espelhamento vertical inverte a imagem no eixo horizontal

No caso de imagens do tipo P2, a matriz *imagem* é percorrida e cada pixel de uma linha é trocado com o pixel da linha simétrica na mesma coluna.

Já no caso de imagens do tipo P3, a matriz *imagemP3* é percorrida e cada pixel (com seus três componentes de cor) é trocado com o pixel da posição simétrica/oposta na mesma coluna, realizando, assim, seu espelhamento vertical.

Ou seja, nesta operação a 1º linha é trocada pela última, a 2º pela penúltima...

5 - Espelhamento Vertical - Exemplos de utilização

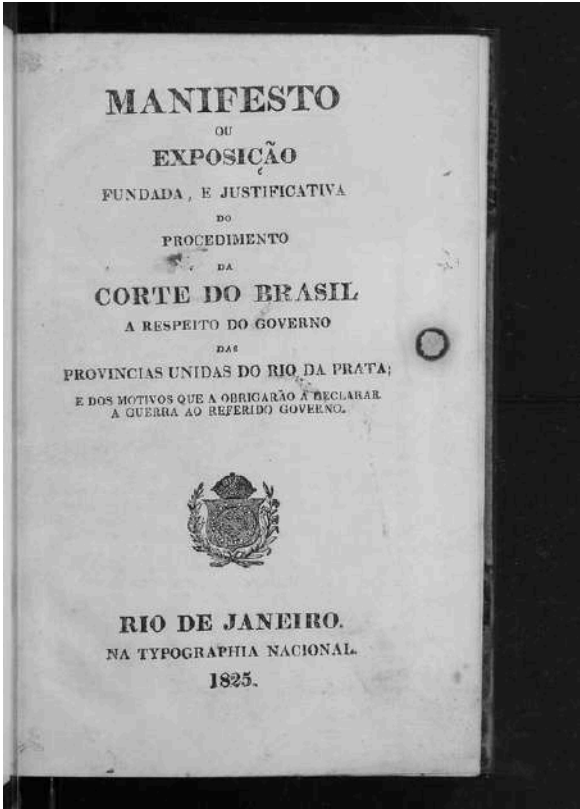


Imagem tipo P2

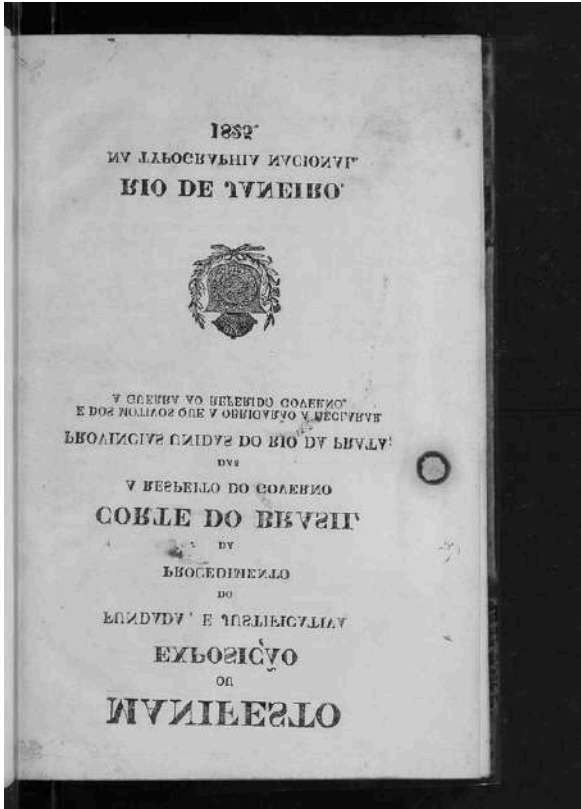
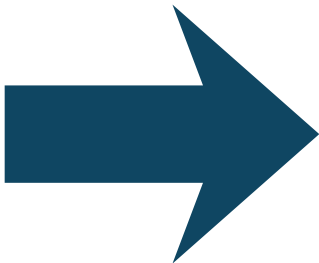


Imagem tipo P2 espelhada verticalmente



Imagem tipo P3

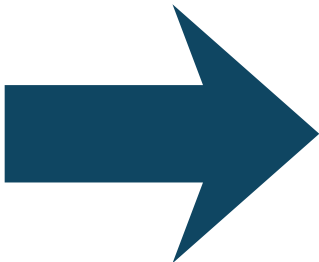


Imagem tipo P3 espelhada verticalmente

6 - Filtro de Sobel

```
**** Aplicar filtro de sobel ****//

else if(options ==6){

    //IMAGEM TIPO P2
    if(strcmp(tipo,"P2")==0) {
        int matgx[3][3] = {1, 0, -1, 2, 0, -2, 1, 0, -1};    // declara as matrizes do filtro sobel
        int matgy[3][3] = {1, 2, 1, 0, 0, 0, -1, -2, -1};
        for(i=0;i<altura+2;i++)                // inicializa a matriz auxiliar com 0
            for(j=0;j<largura+2;j++) {
                matauxP2[i][j] = 0;    //matriz auxiliar criada globalmente
            }
        for(i=0;i<altura;i++)
            for(j=0;j<largura;j++) {
                matauxP2[i+1][j+1] = imagem[i][j]; // copia a imagem original para o centro da matriz auxiliar
            }

        for(i=0;i<altura;i++){                // percorre a matriz 'imagem' novamente para aplicar o filtro
            for(j=0;j<largura;j++){
                int somax = 0;
                int somay = 0;
                int multip;
                for(int t = i;t<i+3; t++){        // percorre os pixels vizinhos e o próprio pixel
                    for(int g = j; g<j+3; g++){
                        multip =(matgx[t-i][g-j])*(matauxP2[t][g]);    // calcula o gradiente horizontal (gx)
                        somax += multip;
                        multip =(matgy[t-i][g-j])*(matauxP2[t][g]);    // calcula o gradiente vertical (gy)
                        somay += multip;
                    }
                }

                int somagp = sqrt((somax*somax)+(somay*somay));    // calcula a magnitude do gradiente
                if(somagp > 255)    // tratamento do caso especial
                    somagp = 255;
                imagem[i][j] = somagp;
                somagp = 0;
            }
        }

        //IMAGEM TIPO P3
        *****
    }
```

Na aplicação do filtro de Sobel, são declaradas as matrizes que representam o filtro, *matgx* e *matgy*, que possuem valores fixos. Também são utilizadas matrizes auxiliares criadas globalmente para realizar a operação de convolução entre matrizes.

Para imagens dos tipos P2 e P3, há diferenças mas o processo segue a mesma lógica: inicialmente, a matriz auxiliar (com bordas) é inicializada com zero. Em seguida, a imagem original é inserida no centro dessa matriz auxiliar.

Após isso, cada pixel é recalculado com base na vizinhança 3x3, nas matrizes do filtro Sobel e nas fórmulas informadas no PDF. O resultado final é armazenado na matriz da imagem (ou imagemP3), gerando a imagem com os valores modificados pelo filtro.

6 - Filtro de Sobel - Exemplos de utilização



Imagem tipo P2

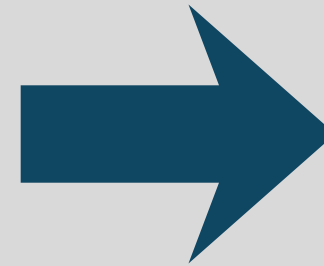


Imagem tipo P2 com filtro de sobel

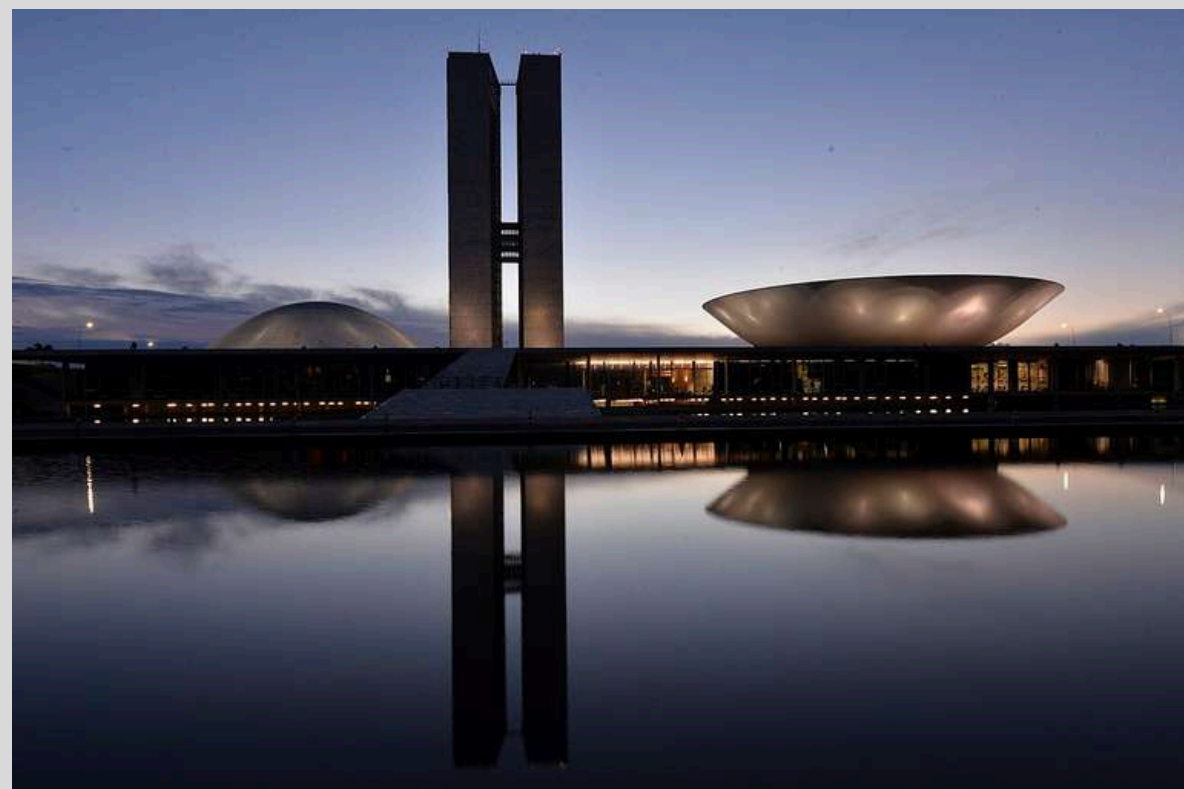


Imagem tipo P3

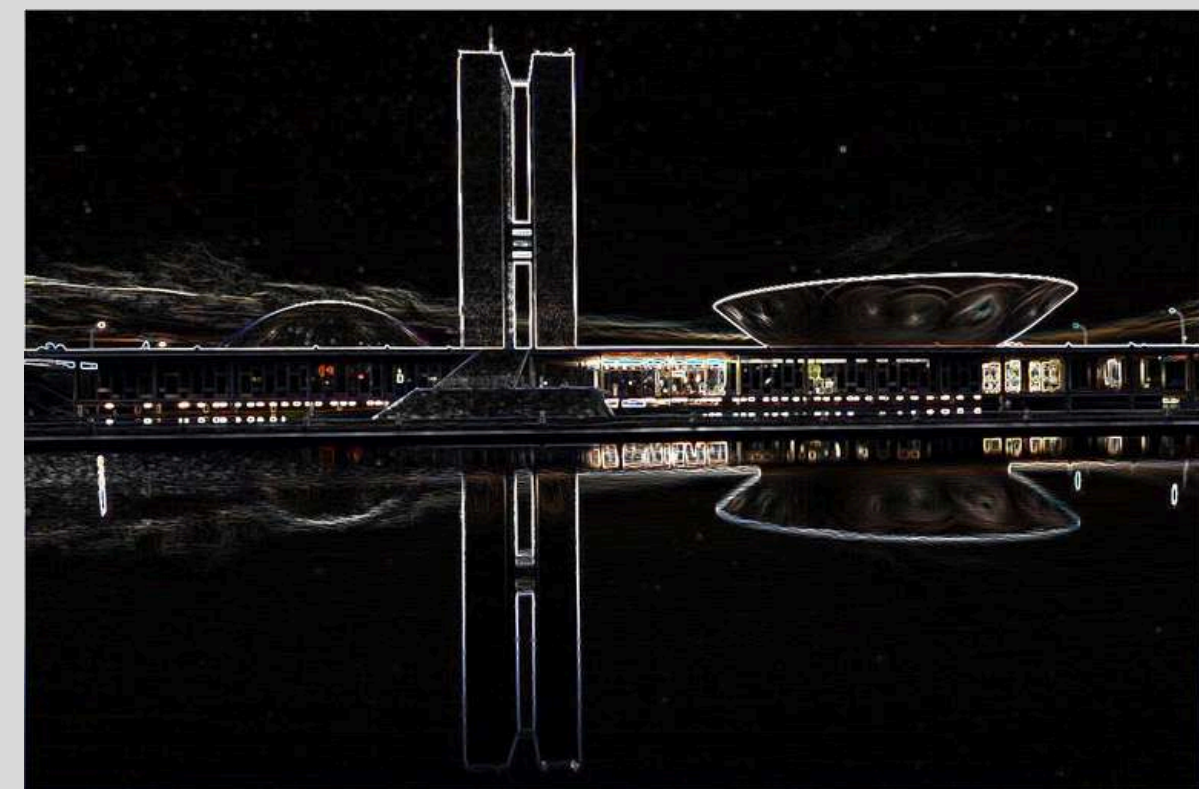
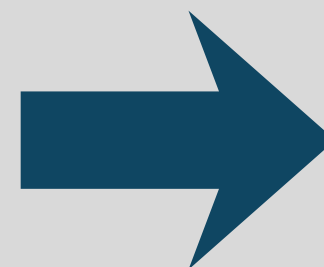


Imagem tipo P3 com filtro de sobel

7 - Filtro de realce

```
/** Filtro de realce */
else if(options ==7){
//IMAGEM TIPO P2
if(strcmp(tipo,"P2")==0) {
    int matfiltro[3][3] = {0, -1, 0, -1, 5, -1, 0, -1, 0}; // declara a matriz do filtro de realce
    for(i=0;i<altura+2;i++) // inicializa a matriz auxiliar com 0
        for(j=0;j<largura+2;j++) {
            matauxP2[i][j] = 0; // matriz auxiliar criada globalmente
        }
    for(i=0;i<altura;i++)
        for(j=0;j<largura;j++) {
            matauxP2[i+1][j+1] = imagem[i][j]; // copia a imagem original para o centro da matriz auxiliar
        }

    for(i=0;i<altura;i++){ // percorre a matriz 'imagem' novamente para aplicar o filtro
        for(j=0;j<largura;j++){
            int soma = 0;
            int multip;
            for(int t = i;t<i+3; t++){ // percorre os pixels vizinhos e o próprio pixel
                for(int g = j; g<j+3; g++){
                    multip =(matfiltro[t-i][g-j])*(matauxP2[t][g]); //calcula o filtro para cada pixel
                    soma += multip;
                }
            }
            if(soma < 0) // tratamento dos casos especiais
                soma = 0;
            if(soma > 255)
                soma = 255;
            imagem[i][j] = soma; //aplica o filtro ao pixel
            soma = 0;
        }
    }
}

//IMAGEM TIPO P3
.....
```

Na operação de filtro de realce, o código realiza um processo semelhante ao utilizado no filtro de Sobel.

Nesse filtro, é declarada a matriz `matfiltro`, responsável por enfatizar detalhes e aumentar a nitidez da imagem.

Para imagens dos tipos P2 e P3, o processo segue a mesma lógica geral.

Primeiramente, a matriz auxiliar (com bordas) é inicializada com zeros. Em seguida, a imagem original é copiada para o centro dessa matriz auxiliar.

Após isso, cada pixel (ou componentes de cor) é recalculado com base em sua vizinhança 3x3, aplicando a matriz de realce por meio da operação de convolução, de forma análoga à aplicação do filtro de Sobel.

7 - Filtro de realce - Exemplos de utilização

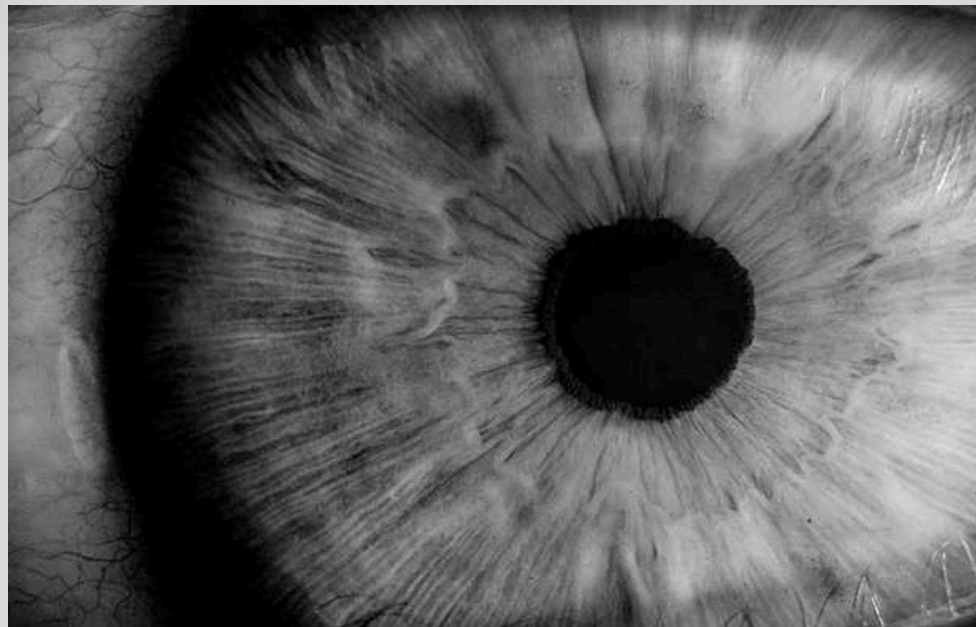


Imagem tipo P2

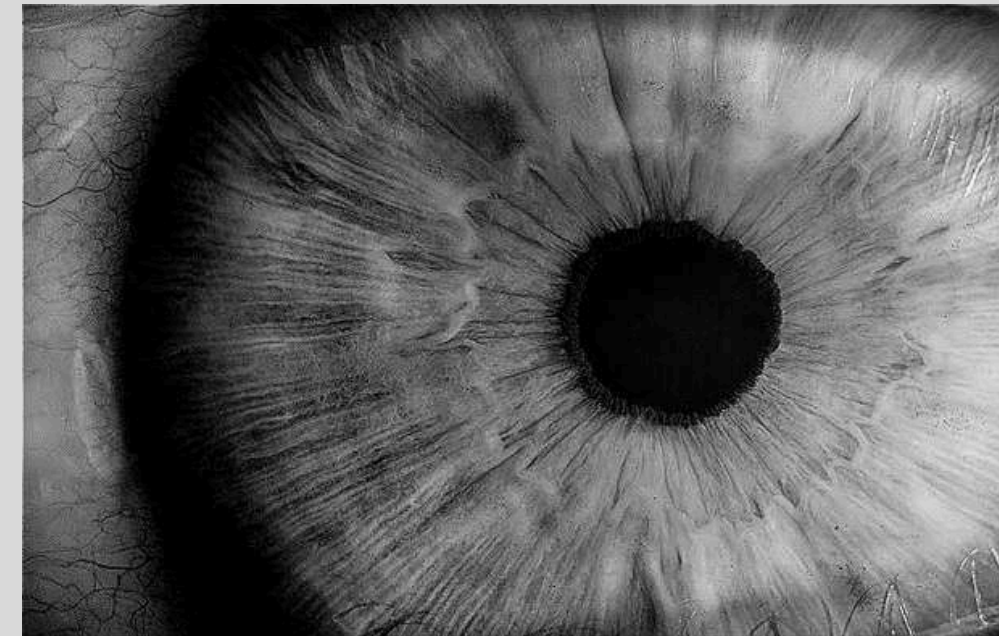
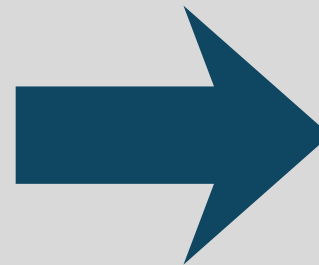


Imagem tipo P2 com filtro de realce

Pelo slide é difícil de notar a diferença. Abrindo o arquivo da foto é possível visualizar a diferença das fotos com mais clareza!



Imagem tipo P3

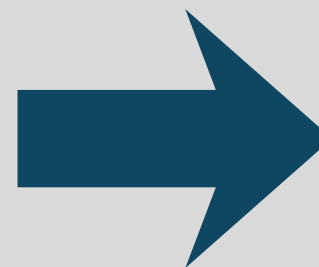


Imagem tipo P3 com filtro de realce

8 - Filtro de binarização

```
/** Filtro de Binarização */
else if(options ==8){

//IMAGEM TIPO P2
if(strcmp(tipo,"P2")==0) {
    for(i=0;i<altura;i++){          //Percorre a matriz 'imagem'
        for(j=0;j<largura;j++){
            if(imagem[i][j] < 127) //Condicional de quando o pixel será 0 ou 255(preto ou branco)
                imagem[i][j] = 0;
            else if(imagem[i][j] >= 127)
                imagem[i][j] = 255;
        }
    }
}

//IMAGEM TIPO P3
else if(strcmp(tipo,"P3")==0) {
    int soma = 0;
    int media = 0;
    for(i=0;i<altura;i++){          //Percorre a matriz 'imagemP3'
        for(j=0;j<largura;j++){
            soma = 0;
            media = 0;
            for(p=0;p<3;p++){
                soma+= imagemP3[i][j][p];    //Soma as bandas de cores de cada pixel
            }
            media = soma/3; //Media da soma das bandas de cores dos pixels

            for(p=0;p<3;p++){ //percorre as bandas novamente para atualizar elas com o novo valor
                if(media < 127) //Condicional de quando o pixel será 0 ou 255(preto ou branco)
                    imagemP3[i][j][p] = 0;
                else
                    imagemP3[i][j][p] = 255;
            }
        }
    }
}
```

Na aplicação do filtro de binarização, cada pixel é transformado em um extremo : preto (0) ou branco (255).

No caso de imagens do tipo P2, a matriz *imagem* é percorrida e, para cada pixel (i, j), é aplicada uma estrutura condicional: se o valor do pixel for menor que 127, ele é transformado em preto; já se o valor for maior ou igual a 127, ele é transformado em branco.

No caso de imagens do tipo P3, a matriz *imagemP3* é percorrida e é calculada a média dos três componentes de cor (R, G e B) de cada pixel.

Em seguida, aplica-se a mesma condição utilizada no caso P2, mas agora no valor da média calculada, para definir se o pixel e seus componentes de cor serão convertidos para preto ou branco.

Tal filtro é usado, por exemplo, por radares de trânsito, que isolam as letras e números escuros contra o fundo branco da placa para identificar veículos.

8 - Filtro de binarização - Exemplos de utilização



Imagem tipo P2

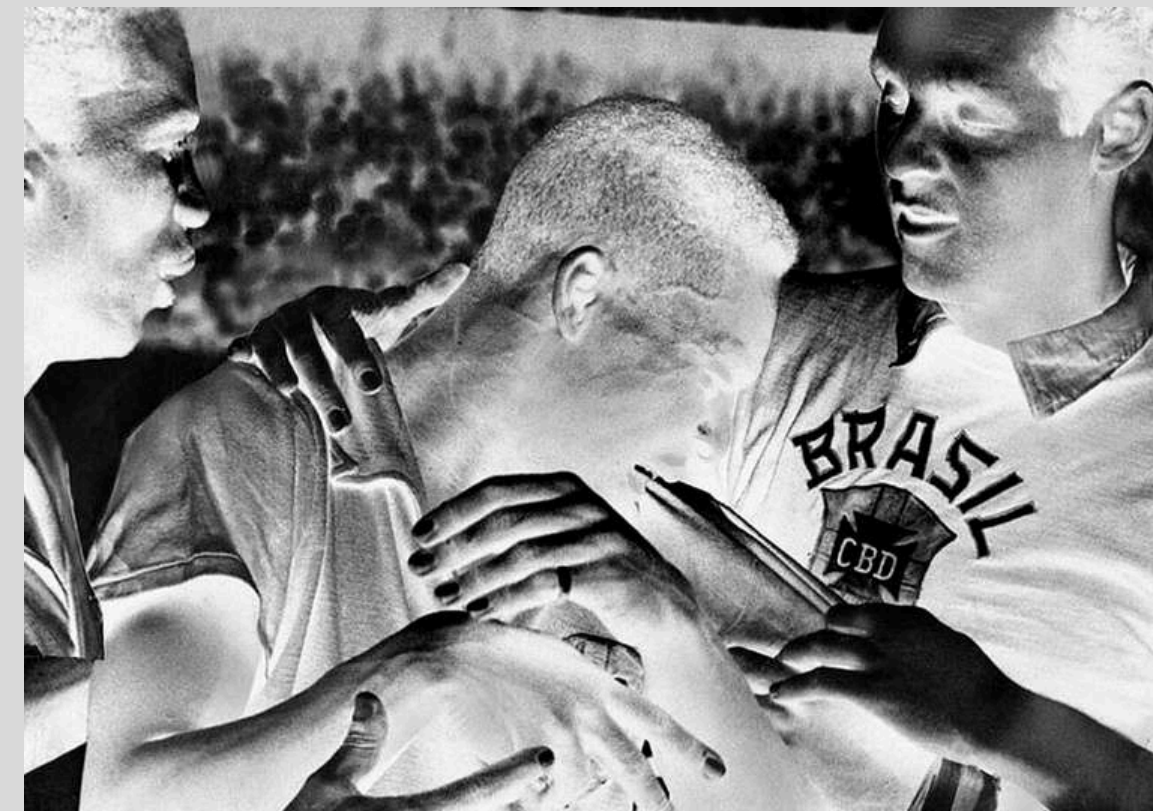
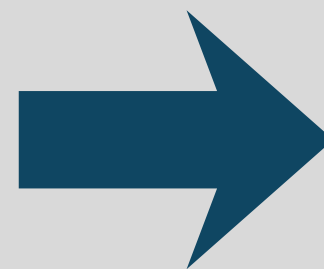


Imagem tipo P2 com filtro de binarização



Imagem tipo P3

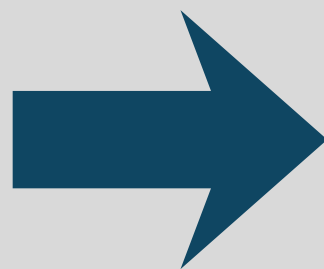


Imagem tipo P3 com filtro de binarização

9 - Colorida para tons de cinza

```
/** Colorida para tons de cinza */
else if(options == 9){

//Executa apenas para imagens do tipo P3
if(strcmp(tipo,"P3")==0) {
    for(i=0;i<altura;i++){ //Percorre a matriz 'imagemP3'
        for(j=0;j<largura;j++){
            // calcula o tom de cinza usando média ponderada dos canais RGB, com valores preestabelecidos
            double cinza = 0.299 * imagemP3[i][j][0] + 0.587 * imagemP3[i][j][1] + 0.114 * imagemP3[i][j][2];
            imagem[i][j] = (unsigned char)cinza; // Salva o resultado na matriz P2

        }
    }
    strcpy(tipo, "P2"); //altera a assinatura da imagem para P2
    cout << "A imagem foi convertida para P2 (tons de cinza)!" << endl;
}
else{
    cout << "Erro: a imagem ja esta em tons de cinza.\n";
    return 0;
}
}

/** FIM DO TRATAMENTO DA IMAGEM */
```

Na operação de transformação de imagem colorida para tons de cinza, esse bloco verifica inicialmente se a imagem é do tipo P3, ou seja, se é uma imagem colorida. Caso contrário, o programa retorna um erro, informando que a imagem já está em tons de cinza.

Se a imagem for do tipo P3, o código percorre a matriz *imagemP3* e calcula o tom de cinza de cada pixel por meio de uma média ponderada dos componentes de cor (R, G e B), utilizando coeficientes preestabelecidos.

O resultado obtido é armazenado na matriz *imagem*, correspondente ao formato P2.

Ao final do processo, a assinatura da imagem é alterada de P3 para P2, indicando que a imagem foi convertida para o formato de tons de cinza.

9 - Colorida para tons de cinza - Exemplos de utilização



Imagem tipo P3

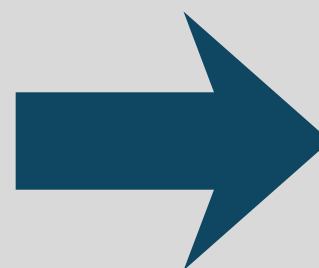


Imagem tipo P2

Gravação da Imagem

```

//*** GRAVACAO DA IMAGEM ***//

// TIPO P2 -----
if(strcmp(tipo,"P2")==0) {

    char nomesaida[100];
    cout << "Digite o nome do arquivo de saida: ";
    cin >> nomesaida;
    arqsaida.open(nomesaida,ios::out); //Abre arquivo para escrita
    if (!arqsaida) {
        cout << "Nao consegui criar novaimagem.pnm\n";
        return 0;
    }

    arqsaida << tipo << endl;    //tipo
    arqsaida << "# TP2-INF110, by RV\n"; //comentario
    arqsaida << largura << " " << altura; //dimensoes
    arqsaida << " " << 255 << endl;    //maior valor
    for(i=0;i<altura;i++){
        for(j=0;j<largura;j++){
            arqsaida << (int)imagem[i][j] << " "; //pixels
        }
        arqsaida << endl;    //Pula a linha
    }
    arqsaida.close();    //fecha o arquivo
}

// TIPO P3 -----
if(strcmp(tipo,"P3")==0) {

    char nomesaida[100];
    cout << "Digite o nome do arquivo de saida: ";
    cin >> nomesaida;
    arqsaida.open(nomesaida,ios::out);    //Abre arquivo para escrita
    if (!arqsaida) {
        cout << "Nao consegui criar novaimagem.pnm\n";
        return 0;
    }

    arqsaida << tipo << endl; //tipo
    arqsaida << "# TP2-INF110, by RV\n";    //comentario
    arqsaida << largura << " " << altura;    //dimensoes
    arqsaida << " " << 255 << endl;    //maior valor
    for(i=0;i<altura;i++){
        for(j=0;j<largura;j++){
            for(p=0; p<3; p++){
                arqsaida << (int)imagemP3[i][j][p] << " "; //espaço em vez de endl
            }
        }
        arqsaida << endl;    // pula linha só quando acaba a largura
    }
    arqsaida.close();    //fecha o arquivo
}
```

Na etapa de gravação da imagem, o programa cria um novo arquivo contendo a imagem após o processamento realizado. Inicialmente, o usuário informa o nome do arquivo de saída e, em seguida, o arquivo é aberto para escrita.

Depois disso, são gravadas as informações do cabeçalho da imagem, como o tipo (P2 ou P3), as dimensões e o valor máximo do pixel (255). Em seguida, os valores dos pixels da matriz imagem ou imagemP3 são percorridos e gravados no arquivo.

Ao final do processo, o arquivo é fechado, concluindo a gravação da nova imagem processada.



..... **Fim da apresentação**

